Local-Ratio Seeding and SCC-Based Refinement for the Minimum Weighted Feedback Arc Set Problem

Soroush Vahidi [ORCID]1*

[1*]Department of Computer Science, New Jersey Institute of Technology, Newark, 07102, NJ, USA.

Corresponding author(s). E-mail(s): sv96@njit.edu;

Abstract

The minimum weighted feedback arc set (MWFAS) problem asks for a minimum-weight set of arcs whose removal makes a nonnegative weighted directed graph acyclic, equivalently for a vertex ordering minimizing total backward arc weight. Despite its NP-hardness, scalable heuristics for sparse weighted digraphs are needed in ranking, dependency repair, and precedence-conflict resolution, where neither exact solvers nor dense tournament algorithms fit the problem structure. This paper presents an algorithm-engineering framework with three components. LR-TA (local-ratio topological add-back) applies iterative cycle-weight reductions to produce a feasible ordering, then recovers forward arcs via topological add-back, reducing unnecessary deletions. A complementary weighted seed provides an independent feasible ordering and a non-worsening baseline. IPSNS (incumbent-protected SCC neighborhood search) applies SCC-local destroy-and-repair refinement, updating the incumbent only when the global backward weight strictly decreases, guaranteeing acyclicity and non-worsening relative to the best internal seed under nonnegative weights.

The contribution is computational and algorithmic rather than a new approximation-ratio theorem. On an exact-validation subset of 57 standard nonnegative instances, the framework matches the bitmask dynamic-programming optimum on 56 of 57, with a mean relative gap of approximately 0.0006%. On 97 standard nonnegative sparse benchmark instances, it achieves the best observed backward weight among all tested methods on 96 instances. Dense LOLIB transfer testing identifies the structural boundary: on complete weighted ordering instances, a matrix-based pairwise-ordering method is stronger, confirming that SCC-local refinement is most effective on sparse directed graphs. A fully reproducible artifact accompanies the paper.

Keywords: Minimum weighted feedback arc set, Combinatorial optimization, Local-ratio algorithm, Strongly connected components, Heuristic search, Algorithm engineering

1 Introduction

The minimum weighted feedback arc set (MWFAS) problem asks for a minimum-weight set of arcs whose removal makes a nonnegative weighted directed graph acyclic, equivalently for a vertex ordering that minimizes the total weight of arcs directed backward with respect to that order. The problem is NP-hard in general [1] and arises across graph optimization, combinatorial ranking, and algorithm engineering wherever pairwise directed relations are cyclic. A directed graph may encode that task u should precede task v, that a circuit signal must be routed before a dependent component is evaluated, that a package must be installed before another can be configured, or that one alternative is preferred to another in paired-comparison data. Rank aggregation from inconsistent pairwise information is a standard motivation for ordering heuristics [2]. In practice, directed relations are often cyclic, arising from noisy measurements, conflicting judgments, reciprocal dependencies, or competing design objectives. When arcs carry weights, backward constraints are not interchangeable: violating a high-weight precedence relation costs more than violating a weak one, and an effective heuristic must be designed around that asymmetry.

Weighted variants of MWFAS have an extensive heuristic and exact-method literature [3, 4]. Exact methods are practical only for small instances or special structures. Naive score heuristics can be fast but ignore local cyclic structure; random multi-start heuristics do not exploit graph decomposition; and specialized dense-ordering solvers may perform well on complete pairwise-comparison instances while being poorly aligned with sparse, irregular directed graphs. A meaningful structural distinction separates sparse general digraphs, where strongly connected components are scattered across a mostly acyclic background, from dense linear-ordering or tournament instances, where every ordered pair carries evidence and the problem is globally coupled. The computational question addressed here is whether a carefully engineered framework built on local-ratio seeding, topological add-back, and SCC-restricted refinement can deliver strong performance on nonnegative sparse weighted digraphs while clearly delineating its scope boundaries.

The local-ratio perspective is an important starting point for feedback arc set heuristics and approximation-oriented reasoning. We do not claim local-ratio as new. Broader local-ratio foundations appear in [5], and directed feedback problems were treated algorithmically in [6]. The contribution in this paper is algorithm-engineering rather than a new approximation theorem: we study a reproducible local-ratio cycle-reduction implementation that records cycle-removal decisions, reconstructs an acyclic topological order, and then performs a topological add-back phase to recover arcs whenever doing so does not reintroduce cyclicity. This method is referred to as local-ratio topological add-back (LR-TA) after the components are defined in Section 4. The emphasis is on deterministic data handling, measurable component effects, and

integration with a refinement stage that is evaluated against exact and external baselines.

The integrated framework combines three components. First, the local-ratio topological add-back procedure produces a feasible ordering from weighted cycle reductions and a reconstruction step. Second, a WMSF-style weighted seed construction provides a complementary initial ordering and internal baseline. Third, incumbent-protected refinement searches within strongly connected components (SCCs). We refer to this refinement as incumbent-protected SCC neighborhood search (IPSNS). IPSNS updates the incumbent only when a candidate ordering has strictly smaller backward weight. This protection gives a simple invariant: the final solution cannot be worse than the best internal seed under the same objective and nonnegative-weight assumptions. The invariant is not an approximation guarantee, but it prevents refinement from degrading the seed and makes the local search behavior traceable instance by instance.

The computational study is organized around five evidence streams. The main sparse benchmark evaluates public weighted directed graph instances derived from graph-benchmark collections, with negative-weight instances separated from the standard nonnegative comparisons. An ablation study isolates the effect of the topological add-back phase, the use of the best seed, the incumbent-protected refinement, and the SCC-priority choice on a representative subset. An exact small-instance validation compares heuristic orderings against a bitmask dynamic-programming optimum where exact optimization is feasible. A sparse external-baseline study compares the proposed framework with the open-source DRMacIver/FAS heuristic, the Eades heuristic as implemented in python-igraph, a weighted adaptation of Eades et al. [7], a Borda net-score ordering, and random multistart. Finally, a dense transfer study evaluates whether the framework carries over to complete weighted linear-ordering instances from the Linear Ordering Problem Library (LOLIB).

The results support a deliberately bounded claim. On 105 sparse benchmark instances, the incumbent-protected refinement has no incumbent violations relative to either internal seed. In the ablation study, the add-back phase reduces mean backward weight by 5.9 percent relative to local-ratio without add-back, and the refinement gives an additional 0.75 percent mean reduction on the tested subset. On the exact-validation subset of 57 standard nonnegative instances with $n \leq 20$, the refinement matches the dynamic-programming optimum on 56 instances, with mean relative gap about 0.0006 percent. On 97 standard nonnegative sparse instances in the external comparison, it achieves the best observed backward weight among the tested methods on 96 instances; the strongest external baseline, DRMacIver, is about 21.6 percent worse in mean backward weight on this sparse benchmark. These numbers support strong performance within the sparse nonnegative weighted-digraph setting studied here.

The dense LOLIB study is reported as a scope boundary rather than as primary evidence. LOLIB instances are complete weighted ordering instances, closely related to the linear ordering problem and structurally different from the sparse directed graphs that motivate the framework. On 50 dense LOLIB instances, the matrix-based pairwise-ordering baseline DRMacIver/FAS obtains the best observed solution on 45

instances, while the proposed refinement is best on 5 instances and has about 3.88 percent higher mean backward weight. This outcome is consistent with the structural distinction: SCC-local refinement is useful when sparse cyclic subgraphs provide meaningful neighborhoods, whereas dense complete ordering instances favor matrix-based pairwise-ordering heuristics. The method is therefore positioned as a reproducible general weighted-digraph framework with strong sparse performance and an explicitly documented dense-ordering limitation.

The contributions of this paper are as follows:

- LR-TA is an engineered local-ratio seed with topological add-back: iterative cycle-weight reductions build a feasible ordering, and a heavy-first arc-recovery phase reduces unnecessary deletions.
- A complementary weighted seed provides an independent feasible ordering and an internal non-worsening baseline, ensuring the refinement begins from a strong alternative constructive solution.
- IPSNS is an incumbent-protected SCC-local destroy-and-repair refinement that concentrates search on strongly connected components with positive backward contribution and accepts a candidate ordering only when the global backward weight strictly decreases.
- The implementation is accompanied by explicit feasibility, termination, and monotonicity statements for LR-TA, add-back, and IPSNS (Section 4.4), including the incumbent-protection invariant without claiming a new approximation ratio.
- An implementation-faithful worst-case complexity characterization is provided for LR-TA, the WMSF-style seed, and IPSNS, expressed in terms of graph size and algorithm parameters such as the IPSNS iteration budget and selected SCC size.
- An extensive computational validation covers exact bitmask dynamic-programming certification on small instances, time-capped MIP validation on medium instances, five external and adapted baselines, a component ablation study, a runtime–budget study, and a dense LOLIB transfer test.
- A fully reproducible artifact accompanies the paper, including all benchmark instances, algorithm implementations, experimental scripts, and saved result summaries.

The remainder of the paper is organized as follows. Section 2 reviews feedback arc set heuristics, local-ratio methods, ordering heuristics, exact methods, and linear-ordering benchmarks. Section 3 defines the minimum weighted feedback arc set objective, the ordering-induced backward weight, and the relation to dense linear-ordering notation. Section 4 describes the local-ratio topological add-back procedure, the weighted seed, and the incumbent-protected SCC refinement. Section 5 presents datasets, baselines, metrics, and reproducibility controls. Section 6 summarizes the computational results. Section 7 discusses limitations, including the dense LOLIB boundary, and Section 8 concludes.

2 Related work

2.1 Directed feedback arc set: formulations and algorithmic foundations

The feedback arc set problem and its weighted variants sit at the intersection of graph optimization, ordering, and ranking. In directed graphs, the problem can be phrased either as deleting a minimum-weight set of arcs to obtain an acyclic graph or as finding an ordering with minimum backward weight. That equivalence makes the problem relevant both to graph-theoretic cycle breaking and to applications that begin from pairwise precedence or preference data. Classical digraph formulations identified minimum feedback arc sets as a central cyclic-structure question [8], early weighted treatments framed the problem as an ordering objective [3], and the general decision problem is computationally hard [1]. The approximation literature is broader for vertex-deletion or undirected variants than for the weighted directed setting considered here, but it still provides important methodological context. [9] study approximation algorithms for minimum feedback sets and multicuts in directed graphs, while [10] analyze tight localizations of feedback sets in more recent algorithm-engineering work. In particular, [5] place local-ratio reasoning in a general approximation framework, showing how weighted combinatorial objectives can be decomposed while preserving feasible improvement structure. Their setting is not the same as minimum weighted feedback arc set on directed graphs, and we do not rely on their theorem to obtain a new approximation guarantee here. However, the paper is directly relevant to the design logic of iterative weight reductions on conflict structures.

For directed feedback problems, [6] provide the closest methodological antecedent to the cycle-reduction component used here. Their contribution is important for two reasons. First, it treats directed feedback problems algorithmically rather than only as ordering formulations. Second, it shows how local-ratio reductions can be applied to directed cycles in a way that is compatible with practical heuristics. The local-ratio step in the present work is inherited in spirit from that line of research: when a directed cycle is found, all arcs on the cycle are reduced by the minimum weight on the cycle, at least one arc becomes tight, and the active graph is simplified. What is new here is not the local-ratio principle itself, but an engineered sparse-graph implementation that records removed arcs explicitly, reconstructs a deterministic topological order once the active graph becomes acyclic, and then runs a heavy-first add-back step to reduce unnecessary deletions.

A systematic comparison of sorting heuristics for feedback arc set appears in [11], who evaluate Eades-family net-score rules, SCC-based decomposition approaches, and other greedy ordering strategies on directed graph instances. Their study provides useful empirical context for the heuristic family: decomposing the problem along SCC boundaries generally improves ordering quality compared with applying net-score rules to the graph as a whole. The present work uses that structural insight as a design principle in both the seed construction and the refinement stage.

The contribution is therefore best understood as an algorithm-engineering framework built around a known reduction principle. The emphasis is on deterministic

data handling, compatibility with sparse weighted digraphs, and integration with a second-stage refinement process that improves only when the global objective decreases.

2.2 Heuristics, library implementations, and order-local search

Heuristic work on feedback arc set and related ordering tasks spans greedy graph rules, local search, and software packages that expose ordering or arc-deletion primitives. A classical baseline is the greedy heuristic of [7]. Their method is attractive because it is fast, interpretable, and structurally simple: it repeatedly extracts sources, sinks, or high net-score vertices to construct an order. An earlier denser-graph variant in the same heuristic family appears in [12]. Even when a modern study ultimately outperforms such a heuristic, it remains an essential reference point because many later practical systems either implement that rule directly or build on the same score-based intuition. In the present paper, Eades-style ideas appear in two places: as the historical baseline that motivates fast greedy ordering, and as the conceptual starting point for external and adapted software baselines used in the experiments.

The external-baseline design intentionally mixes library-grade and specialized open-source tools. The python-igraph documentation for `Graph.feedback_arc_set` [13] exposes both an Eades-style heuristic and exact integer-programming modes. It is a useful baseline because it reflects what a practitioner could call from a general-purpose graph library. In contrast, the DRMacIver/FAS tool [14] is a matrix-based pairwise-ordering heuristic rather than a general graph-analysis package. Its documentation formulates the input as a nonnegative $n \times n$ matrix W, where W_{ij} represents evidence that item i should precede item j, and seeks a permutation maximizing the total forward weight $\sum_{p_i<p_j} W_{ij}$. The repository describes the method as deterministic and reports a local-optimality property with respect to single-element moves; it does not present the tool as an exact optimizer. Because the documented model is matrix-based and has $O(n^2)$ complexity in the number of items, it is especially natural as a dense pairwise-ordering baseline, while remaining usable in our experiments through the sparse entry-list interface. Empirically, it is the strongest external baseline on the sparse standard benchmark and the dominant method on the dense LOLIB transfer test.

The remaining baselines in the experiments are deliberately simple and interpretable. Weighted net-score ordering and random multistart are not intended to represent state of the art; they provide calibration points. Net-score and win-based tournament rules are classical ranking ideas with theoretical support in the weighted tournament literature [15]. The weighted Eades adaptation used here should also be interpreted carefully. The original Eades–Lin–Smyth paper [7] does not define the weighted adaptation used in this study, so that baseline is described as a local adaptation rather than as the original authors' algorithm. That distinction is part of the methodological hygiene of the study: external baselines should be credited as external when they are external, and local adaptations should be labeled as such even when they are inspired by well-known methods.

Beyond greedy seeds, simpler order-local search strategies provide useful methodological context. Adjacent-swap local search and single-vertex insertion local search

represent the class of moves that improve an ordering by reordering one or two elements at a time without reference to SCC structure. A natural question for any SCC-aware refinement framework is whether such moves, seeded from a strong constructive heuristic, can replicate its gains. The study includes a direct comparison in EXP7 to determine whether these order-local moves suffice, allowing the contribution of SCC-specific structure to be isolated from generic greedy improvement.

There is a broader family of learning-based ranking methods that could be discussed without entering the experimental comparison set. GNNRank [16], for example, frames ranking from pairwise comparisons as a trainable graph-neural ranking problem. It is relevant as a signal that the ranking literature has moved beyond purely combinatorial heuristics, but it is not used as a baseline here because the present study focuses on deterministic, training-free graph heuristics rather than learned ranking models. Recent weighted feedback arc set heuristics such as the minimal-and-stable construction of Cavallaro and Cutello [17] are likewise outside the comparison set because they were not rerun for this study, but they illustrate continued heuristic interest in weighted feedback arc deletion.

2.3 Exact methods and computational hardness

The exact side of the literature serves two different roles in this paper. First, it sets expectations about computational hardness. Second, it provides the reference standard for the small-instance validation experiment. [4] give a modern exact treatment of minimum feedback arc set through formulations that connect the problem to broader combinatorial optimization machinery. Their work is not the computational baseline for the full sparse benchmark in this paper; the sparse benchmark is too large for exact optimization to be the main comparison mode. Instead, exact methods enter the study through a limited validation regime on small instances, where the question is not whether exact optimization scales globally, but whether the heuristic framework is near-optimal when exact answers are available.

This distinction also helps separate feedback arc set from the larger family of ordering models. Dense linear ordering formulations are often written as forward-weight maximization on complete matrices, whereas sparse directed graph formulations arise more naturally as backward-weight minimization on incomplete digraphs. The two are mathematically linked, but they are not computationally identical in practice.

The present study deliberately moves between the two views. The sparse benchmark is the main claim-bearing environment and is presented in graph terms. The LOLIB transfer test is presented in linear-ordering terms because those instances are dense complete ordering problems. Keeping both formulations visible is important. Otherwise the reader could mistakenly treat good sparse-graph performance as evidence of dense linear-ordering dominance, which the experiments do not support.

At larger scales, exact computation is often replaced by decomposition, approximation, or specialized engineering. [18] provide one useful point of reference by treating feedback arc set computation in a web-scale setting. Tournament and dense-ordering settings have their own fast heuristic literature, including local-search methods for weighted tournament feedback arc set [19] and fixed-parameter approaches for the unweighted tournament case [20]. Their work is not directly comparable to the exact

small-instance validation in this paper, but it illustrates the broader computational reality: scalable feedback arc methods are usually driven by structure, heuristics, or deployment constraints rather than by globally exact optimization. The goal here is not to displace exact methods; it is to present a heuristic framework whose computational behavior is strong on the sparse benchmark and whose limitations on dense complete ordering instances are reported explicitly.

2.4 Dense linear ordering and tournament methods as a structurally distinct regime

Dense linear ordering and tournament methods constitute a structurally distinct computational regime from the sparse weighted-digraph setting studied here, and the dense-transfer side of the paper is best understood in that context. The dense-transfer side of the paper is best understood through the linear ordering literature. [21] provide a benchmark-oriented account of the Linear Ordering Problem Library (LOLIB) and its heuristic evaluation culture. That paper is the most appropriate formal reference for LOLIB-style benchmark usage here because it anchors the library in a published benchmark-comparison setting rather than only as a download page. The official LOLIB page itself [22] remains important for provenance and dataset identification, especially because benchmark users often obtain the instances directly from library pages or their mirrors. Together, these sources explain why dense complete tournaments have a different methodological status from sparse DIMACS-style directed graphs: they are not merely larger graphs of the same type, but instances drawn from a different benchmark tradition with different algorithmic expectations.

The sparse benchmark side has a different provenance. The graph-benchmarks collection [23] is a public set of directed instances in DIMACS-style format rather than a linear-ordering library.

That difference supports a central design choice: the sparse and dense benchmarks test different claims. The sparse benchmark is the main evidence for practical value on nonnegative weighted digraphs. The dense benchmark is a transfer test used to determine where a general weighted-digraph heuristic stops being competitive against dense-ordering-native methods.

The ranking-from-pairwise-comparisons viewpoint connects these benchmark families conceptually. In dense complete settings, every ordered pair carries evidence, so forward-weight maximization aligns naturally with tournament and ranking formulations [2, 15]. In sparse settings, many pairwise relations are absent, heterogeneous, or structurally constrained by applications such as dependency repair or precedence resolution.

This is exactly why a method that performs well on sparse graph-benchmarks may still lose on LOLIB. The issue is not inconsistency in the experiments; it is a change in problem structure. Related work therefore has to prepare the reader for a comparative message that is strong but bounded.

This paper occupies a specific position in the literature. It is not a new approximation-theory paper in the style of local-ratio foundations [5] or directed-feedback algorithm design [6, 9]. It is not an exact-method paper in the style of [4].

It is not a web-scale systems paper in the style of [18]. It is also not a dense linear-ordering or trainable ranking paper in the style of the LOLIB benchmark tradition [21], tournament local search [19], or graph-neural ranking work such as GNNRank [16]. Instead, it is an integrated computational heuristic study whose main object is a reproducible framework for nonnegative weighted directed graphs.

That framework combines three roles that are often separated in the literature: a cycle-reduction seed grounded in prior local-ratio ideas, a complementary weighted seed, and incumbent-protected SCC neighborhood refinement. IPSNS is the main algorithmic contribution beyond the engineered LR-TA seed. It concentrates search on strongly connected components with positive backward contribution, applies SCC-local destroy-and-repair moves with restricted local-ratio repair, and accepts a candidate only when the global backward weight strictly decreases. The practical significance of that combination is twofold. First, it gives a transparent explanation for why the method should be robust on sparse directed graphs: the initial seeds construct feasible orders in different ways, and the refinement concentrates effort on strongly connected regions where ordering uncertainty remains. Second, it yields a claim that is strong enough to matter and narrow enough to defend: the framework is highly effective on the main sparse benchmark, near-optimal on the exact-validation subset, and explicitly limited on dense LOLIB instances where the external matrix-based pairwise-ordering baseline is stronger.

3 Problem definition

Let $G = (V, A, w)$ be a directed graph with vertex set V, arc set $A \subseteq V \times V$, and nonnegative arc weights $w : A \to \mathbb{R}_{\geq 0}$. A feedback arc set is a set $F \subseteq A$ such that removing F makes the graph acyclic. The minimum weighted feedback arc set problem asks for a minimum-weight feedback arc set, i.e., a set F minimizing $\sum_{a \in F} w(a)$.

An ordering π assigns each vertex a distinct position. An arc (u, v) is backward under π if u appears after v. The ordering-induced backward weight is

$$\mathrm{bw}(\pi) = \sum_{(u,v) \in A:\ \pi(u) > \pi(v)} w(u, v). \tag{1}$$

For any ordering π, the set of backward arcs is a feedback arc set: all remaining forward arcs point from earlier to later positions and therefore form an acyclic graph. Conversely, every acyclic subgraph admits a topological order. The weighted feedback arc set problem can therefore be treated as the problem of finding an ordering with minimum backward weight [1, 3].

Dense linear ordering problem benchmarks are often stated in terms of a complete weight matrix C, where an ordering $\pi = (\pi_1, \ldots, \pi_n)$ is evaluated by the forward weight. In that notation,

$$\mathrm{fw}_C(\pi) = \sum_{1 \leq i < j \leq n} C_{\pi_i, \pi_j}, \qquad \mathrm{bw}_C(\pi) = \sum_{1 \leq i < j \leq n} C_{\pi_j, \pi_i} = W_C - \mathrm{fw}_C(\pi), \tag{2}$$

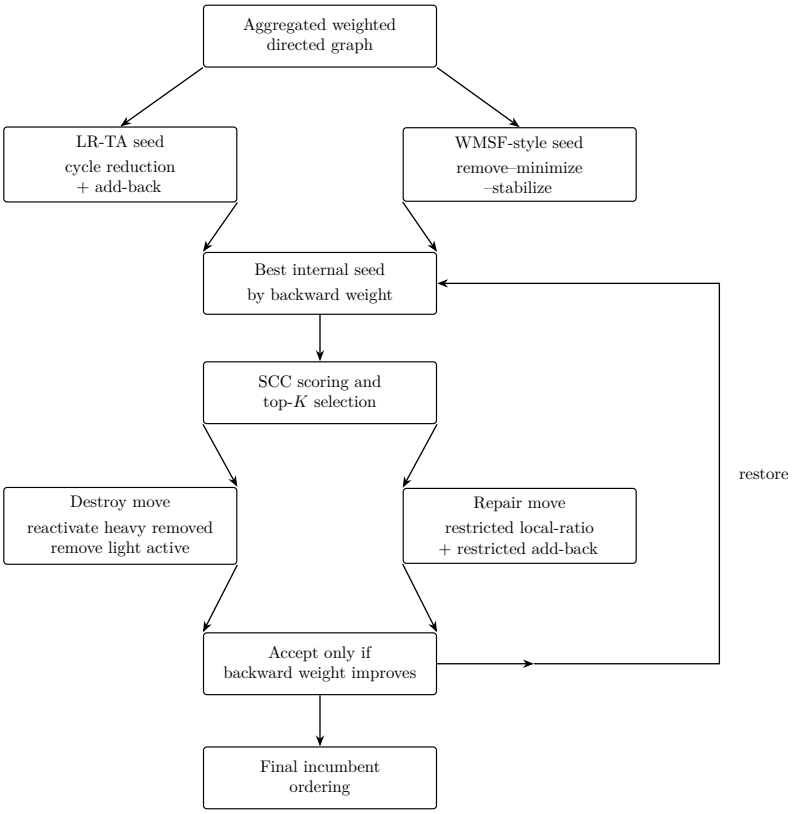

Fig. 1: Conceptual overview of the integrated framework. Two constructive seeds are evaluated first; IPSNS then applies incumbent-protected SCC-local destroy-and-repair moves to the better seed only when they improve the global objective.

where $W_C = \sum_{i \neq j} C_{ij}$ is the total off-diagonal weight. Thus maximizing forward weight on a complete dense linear-ordering instance is equivalent to minimizing backward weight after converting the matrix to a complete weighted digraph. LOLIB results are reported as a transfer test. Those instances are dense complete ordering problems rather than sparse directed graph benchmarks.

4 Proposed methodology

The framework combines three layers: a local-ratio topological add-back seed, a complementary weighted seed, and an incumbent-protected refinement over strongly connected components. Figure 1 summarizes the data flow, while Tables 1 and 2 isolate the roles of each component and the invariants that the implementation preserves.

The implementation reads an aggregated weighted DIMACS digraph and stores it in edge-indexed arrays. Each arc receives an edge identifier together with fixed

Table 1: Algorithm components and their roles in the framework.

Component	Primary mechanism	Role in this paper
LR-TA	Global local-ratio cycle reduction followed by heavy-first add-back	Engineered reproducible seed; main constructive method derived from prior local-ratio ideas
WMSFstyle seed	Ordered arc removal, minimization, stabilization, and re-minimization inside SCCs	Complementary internal seed and baseline; broadens incumbent quality before refinement
IPSNS	SCC-local destroy–repair moves with restricted local-ratio repair and restricted add-back	Main refinement framework; improves only when the global objective decreases

Table 2: Implementation-level invariants preserved by the framework.

Property	Meaning
Acyclic seed output	LR-TA and the WMSF-style seed both output an acyclic active graph that induces a valid ordering.
Heavy-first add-back	Reinserted edges are processed in nonincreasing original weight, so high-weight arcs are given priority for restoration.
Deterministic topological order	Kahn-style topological reconstruction uses deterministic tie-breaking, making seed output reproducible for fixed input.
Incumbent protection	IPSNS updates the incumbent only when the candidate backward weight is strictly smaller.
Reproducible randomization	IPSNS SCC selection and destroy fractions are fully determined by the configured random seed.

endpoint arrays and an activity flag. This design is shared across LR-TA, WMSF-style seeding, and IPSNS. The common objective is the backward weight defined in Section 3; all moves are evaluated under that objective, and all determinism claims in this section refer to fixed input graphs and fixed random seeds.

4.1 Local-ratio topological add-back

LR-TA is the first constructive component. It inherits the idea of local-ratio cycle reduction from the directed-feedback setting of [6], but the implementation studied here is an engineered sparse-graph instantiation rather than a new theorem. The algorithm maintains an active subgraph over the original edge identifiers and repeatedly applies two phases.

In Phase I, the algorithm searches the active graph for any directed cycle using an iterative depth-first search. The search is deterministic because vertices and outgoing edge lists are scanned in fixed order, and only the first detected cycle is used. If C is

the detected cycle, LR-TA computes

$$\varepsilon(C) = \min_{e \in C} w(e) \tag{3}$$

and reduces the mutable weight of every arc in C by $\varepsilon(C)$. At least one arc becomes tight. Any arc whose reduced weight falls below the numerical tolerance is deactivated and appended to the removed-edge set F_{LR}. The phase terminates when no directed cycle remains in the active graph, so the active subgraph is a directed acyclic graph.

In Phase II, LR-TA attempts to reinsert removed arcs without recreating cyclicity. Removed arcs are processed in nonincreasing order of original weight, with deterministic tie-breaking on endpoints and edge identifier. Let π be a topological order of the current active directed acyclic graph and let $\mathrm{rank}_\pi(v)$ denote the position of vertex v. A candidate arc (u, v) is accepted immediately if

$$\mathrm{rank}_\pi(u) < \mathrm{rank}_\pi(v), \tag{4}$$

because the current topological order already certifies that the insertion is forward. Otherwise the algorithm checks whether v can reach u in the current active graph, using a reachability search pruned by the topological rank interval. The arc is reinserted if and only if no such path exists. Whenever a backward candidate is accepted, the topological order is recomputed so that future admissibility tests remain exact. The extracted topological order is deterministic under the implementation used here, but it is not mathematically unique: different linear extensions of the same DAG can exist. The chosen extraction rule is therefore an engineering convention rather than a canonical ordering of an acyclic state.

Algorithm 1 summarizes the implementation.

The add-back phase is where this implementation departs most clearly from a bare local-ratio reduction. It is also the component with the strongest ablation signal in the current experiments. LR-TA should therefore be understood as a two-phase engineered seed rather than as a one-step local-ratio routine.

4.2 WMSF-style seed construction

The second seed is a weighted removeArcs–minimize–stabilize pipeline implemented locally as an internal baseline. It broadens the seed space available to the refinement stage and helps define the incumbent-protection guarantee.

At a high level, the WMSF-style seed decomposes the graph into strongly connected components and processes each nontrivial component separately. Within a component, it first removes arcs according to a prescribed weight-based ordering. Two orderings are supported in code: $L1$, which sorts arcs primarily by weight, and $L2$, which normalizes arc weight by a weighted in/out-degree denominator. The resulting removed set is then minimized by heavy-first add-back, followed by a stabilization step and a second minimization pass. When the entire graph is a single nontrivial strongly connected component, both $L1$ and $L2$ are tried and the lower-backward-weight seed is kept. Otherwise the configured ordering is used componentwise.

Algorithm 1 LR-TA seed construction

Require: Weighted digraph $G = (V, A, w)$ with $w \geq 0$
Ensure: Ordering π_{LR}, removed-edge set F_{LR}
Build edge-indexed arrays $(U, V, W_0, W, \mathit{active}, \mathit{adj})$
$F_{\mathrm{LR}} \leftarrow \emptyset$
while the active graph contains a directed cycle C **do**
    $\varepsilon \leftarrow \min_{e \in C} W[e]$
    for all $e \in C$ **do**
        $W[e] \leftarrow W[e] - \varepsilon$
        if $W[e] \leq \tau$ and e is active **then**
            deactivate e; set $W[e] \leftarrow 0$; add e to F_{LR}
        end if
    end for
end while
Compute deterministic topological order π of the active graph
for all $e = (u, v) \in F_{\mathrm{LR}}$ in nonincreasing order of $W_0[e]$ **do**
    if $\mathrm{rank}_\pi(u) < \mathrm{rank}_\pi(v)$ **then**
        reactivate e; remove e from F_{LR}
    else if $v \not\rightsquigarrow u$ in the current active graph **then**
        reactivate e; remove e from F_{LR}
        recompute deterministic topological order π
    end if
end for
return $\pi_{\mathrm{LR}}, F_{\mathrm{LR}}$

The point of carrying this seed into IPSNS is not to establish a separate literature claim about WMSF. Instead, it provides a complementary structural bias to LR-TA. LR-TA is driven by cycle reductions in the active graph. The WMSF-style seed is driven by ordered arc removals and subsequent minimization. Because the two constructions fail differently on some instances, taking the better of them before refinement produces a stronger and more robust incumbent than either seed alone.

IPSNS is the main refinement contribution. It combines SCC-local destroy-and-repair moves, restricted local-ratio repair, and restricted heavy-first add-back with a strict incumbent-protection rule. Refinement is therefore concentrated on cyclic regions where ordering uncertainty remains, while the global objective can only improve or stay unchanged.

4.3 Incumbent-protected SCC-neighborhood search

IPSNS begins by computing both seeds on the full graph: the full WMSF-style seed and the LR-TA seed. Their backward weights are evaluated under the common ordering objective, and the better one becomes the initial incumbent. The refinement stage then iterates over strongly connected components of the original aggregated graph, but the selection is filtered by the incumbent ordering: only components with positive current backward contribution are eligible, and a weighted-random choice is drawn

from the top-K contributors by SCC backward weight. This makes the neighborhood search concentrated rather than exhaustive.

Within the chosen component, IPSNS applies a destroy-and-repair move. The destroy stage has two parts. First, a fraction of currently removed internal edges is reactivated in heavy-first order, temporarily giving the neighborhood more freedom. Second, a small fraction of currently active internal edges is forcibly removed in light-first order, encouraging escape from a locally stable order. The repair stage then applies a restricted local-ratio cycle reduction on that SCC neighborhood and finishes with restricted heavy-first add-back minimization. All graph edits are confined to the chosen component's internal edges, while the rest of the incumbent graph remains untouched during the move.

If the repaired candidate fails to admit a valid topological order, the entire SCC-local state is reverted. If the candidate is valid, the full incumbent objective is recomputed. The incumbent is updated only if the new backward weight is strictly smaller than the current best value. Otherwise the move is rejected and the previous incumbent is restored exactly. This is what makes the method incumbent-protected: refinement never commits a worsening move.

Algorithm 2 summarizes the IPSNS driver.

Two implementation details are especially important for interpretation. First, the default WMSF seed mode inside IPSNS is the full per-SCC pipeline, not a weaker legacy approximation; this is required so that the refinement never starts from a seed weaker than the standalone WMSF baseline. Second, the randomization in IPSNS is controlled entirely by a fixed seed. The resulting sequence of SCC choices and destroy fractions is therefore reproducible given the same input and parameter settings. The parameter choices themselves are interpreted conservatively later in the paper: the ablation study supports them as reasonable defaults on the tested subset, not as universally optimal settings.

4.4 Correctness, termination, and computational complexity

This subsection states the feasibility, termination, monotonicity, and complexity properties supported by the implementation in `src/mwfas/`. Throughout, let $n = |V|$, $m = |A|$ after aggregation, and let $\tau > 0$ denote the numerical tolerance used in code (default 10^{-12}). For LR-TA, let c be the number of Phase I cycle-reduction iterations and $r = |F_{\mathrm{LR}}|$ the number of removed arcs processed in Phase II add-back. For IPSNS, let T be the iteration budget and K the top-K SCC pool size. For a selected SCC S, write $n_S = |V(S)|$, $m_S = |A(S)|$, and let c_S and r_S denote the numbers of restricted cycle-reduction rounds and restricted add-back candidates inside S during one destroy-and-repair move.

All bounds below are worst-case upper bounds for the implemented procedures. Practical runtime depends on graph density, SCC structure, the number of detected cycles, how often backward add-back triggers reachability search or topological recomputation, and the configured IPSNS budget.

Algorithm 2 IPSNS with incumbent protection

Require: Weighted digraph $G = (V, A, w)$, iteration budget T
Ensure: Ordering $\pi^\star$ with objective no worse than the best internal seed
Build WMSF-style seed π_{W} and LR-TA seed π_{LR}
$\pi^\star \leftarrow \arg\min\{\mathrm{bw}(\pi_{\mathrm{W}}), \mathrm{bw}(\pi_{\mathrm{LR}})\}$
for $t = 1, \ldots, T$ **do**
    Score each nontrivial SCC by its current backward contribution
    if all SCC scores are zero **then**
        break
    end if
    Sample one SCC from the top-K positive-score SCCs using weighted random choice
    Save the current internal SCC state
    Reactivate a heavy fraction of removed internal edges
    Remove a light fraction of active internal edges
    Apply SCC-restricted local-ratio repair
    Apply SCC-restricted heavy-first add-back minimization
    if the candidate SCC state is invalid **then**
        restore saved SCC state; **continue**
    end if
    Recompute the global ordering and candidate objective
    if the candidate objective is smaller than $\mathrm{bw}(\pi^\star)$ **then**
        accept the candidate and update $\pi^\star$
    else
        restore the previous incumbent exactly
    end if
end for
return $\pi^\star$

Proposition 1 (LR-TA feasibility and termination) *Assume nonnegative original weights and the edge-indexed active-graph representation used in* `local_ratio_fas_fast`*. Then LR-TA satisfies the following properties.*

1. ***Cycle reduction progress.*** *In each Phase I iteration on a detected directed cycle C, the implementation subtracts $\varepsilon(C) = \min_{e \in C} W[e]$ from every arc of C. If $\varepsilon(C) > \tau$, at least one arc on C reaches weight $\leq \tau$ after the subtraction and is deactivated; if $\varepsilon(C) \leq \tau$, one arc of C is deactivated directly.*
2. ***Phase I termination.*** *Phase I executes at most m iterations, because each iteration deactivates at least one active arc.*
3. ***Acyclic output of Phase I.*** *When Phase I stops, the active subgraph contains no directed cycle and therefore admits a topological order.*
4. ***Add-back feasibility.*** *Phase II reactivates an arc only by the forward-rank shortcut (Equation (4)) or after a negative reachability test certifying that reactivation preserves acyclicity; accepted backward restorations trigger recomputation of the topological order before later tests.*

5. ***Valid ordering.*** *The final topological order of the active graph together with the remaining removed set defines a feasible feedback arc set under the ordering objective of Section 3.*

Proof Items (1)–(2) follow directly from the update rules in Phase I: every iteration while a cycle exists either subtracts the minimum cycle weight, which makes at least one cycle arc tight under exact arithmetic and deactivates every arc whose mutable weight falls to $\leq \tau$, or, when $\varepsilon(C) \leq \tau$, deactivates one cycle arc explicitly. Since the number of active arcs is finite and decreases by at least one per iteration, Phase I performs at most m iterations. Item (3) is the stopping condition of the cycle-detection loop. Items (4)–(5) follow from Proposition 2 and the final call to deterministic topological sorting on the acyclic active graph. □

Proposition 2 (Correctness of the add-back shortcut) *Let $G' = (V, A')$ be the current active acyclic graph with topological order π and rank function rank_π. For a removed arc (u, v):*

1. *If $\mathrm{rank}_\pi(u) < \mathrm{rank}_\pi(v)$, reactivating (u, v) cannot create a directed cycle in G'.*
2. *If $\mathrm{rank}_\pi(u) > \mathrm{rank}_\pi(v)$ and there is no directed path $v \rightsquigarrow u$ in G', then reactivating (u, v) cannot create a directed cycle.*
3. *If a backward arc is reactivated under item (2), recomputing a topological order before subsequent add-back tests preserves the validity of later reachability and rank tests.*

The same three rules are used in LR-TA Phase II, WMSF-style minimization, and SCC-restricted add-back inside IPSNS.

Proof For (1), the reactivated arc is forward with respect to π, so any directed cycle containing it would require a directed path from v to u using only previously active arcs; such a path would already imply $\mathrm{rank}_\pi(v) < \mathrm{rank}_\pi(u)$, contradicting the forward-rank condition. For (2), if no path $v \rightsquigarrow u$ exists, adding (u, v) cannot close a directed cycle, because every cycle through the new arc would require such a path. For (3), subsequent tests are applied with respect to the updated acyclic active graph; recomputing π after a backward insertion is exactly what the implementation does before processing later removed arcs. □

Proposition 3 (IPSNS feasibility and monotonicity) *Let $\pi^{(0)}$ be the better of the two internal seeds and let $\pi^{(t)}$ denote the incumbent ordering after iteration t of IPSNS. Then:*

1. *Every accepted candidate admits a valid global topological order and therefore a feasible ordering.*
2. *If a destroy-and-repair move produces an invalid SCC-local state, or if the repaired global ordering is not acyclic, the implementation reverts to the pre-move incumbent.*
3. *If the repaired candidate is feasible but does not strictly improve backward weight, the incumbent is restored exactly.*

4. Consequently,

$$\mathrm{bw}\Big(\pi^{(t+1)}\Big) \le \mathrm{bw}\Big(\pi^{(t)}\Big) \le \mathrm{bw}\Big(\pi^{(0)}\Big) \qquad \textit{for all } t \ge 0.$$

This is a monotonicity guarantee on the incumbent sequence, not an approximation-ratio or local-optimality guarantee.

Proof The incumbent is initialized from the better seed, so $\mathrm{bw}(\pi^{(0)})$ is well defined. At each iteration, the algorithm either rejects the move and leaves the incumbent unchanged, or accepts a candidate only after (i) successful SCC-local repair, (ii) successful global topological sorting, and (iii) strict improvement $\mathrm{bw}(\pi) < \mathrm{bw}(\pi^\star)$. Rejection therefore leaves the objective unchanged; acceptance strictly decreases it. The stored snapshot guarantees that the final returned ordering is never worse than the best seed objective value. □

Proposition 4 (IPSNS termination) *IPSNS terminates after at most T iterations. An iteration may terminate early if every nontrivial SCC has zero backward contribution under the current incumbent. Each iteration performs finitely many graph operations on the selected SCC and on the full graph for objective evaluation.*

Proof The outer loop in `lns_merge_wmsf_lr_best_incumbent` is bounded by the finite parameter T. Inside an iteration, SCC scoring, destroy, restricted cycle reduction, restricted add-back, and one global backward-weight evaluation each traverse finite edge sets. Early termination occurs only when the filtered SCC list is empty. □

LR-TA time complexity.

Phase I performs at most m iterations. Each iteration calls iterative DFS cycle detection `find_any_cycle_eids`, which scans adjacency lists and touches each active edge only a constant number of times per search in the worst case, giving cost $O(n+m)$ per iteration and

$$O\big(m(n+m)\big)$$

overall for Phase I. Phase II sorts at most r removed arcs in $O(r \log r)$, then processes each removed arc once. A forward-rank acceptance is $O(1)$; a backward candidate may trigger one pruned reachability test and, if accepted, one topological recomputation, each costing $O(n+m)$ in the worst case. Phase II therefore costs

$$O\big(r \log r + r(n+m)\big).$$

In practice, forward shortcuts and rank-bounded reachability often reduce the number of full traversals.

WMSF-style seed time complexity.

The seed first computes SCCs by Kosaraju in $O(n+m)$. For each nontrivial SCC with k vertices and m_S internal arcs, the pipeline `removeArcs` → `MinimizeFas` → `StabilizeFas` → `MinimizeFas` performs:

- arc sorting and ordered deletion in $O(m_S \log m_S)$, with periodic acyclicity checks each costing up to $O(k + m_S)$;
- two heavy-first add-back passes with the same structure as LR-TA Phase II on the SCC, hence up to $O(r_S \log r_S + r_S(k + m_S))$ per pass;
- up to $O(\log k)$ stabilization passes, each scanning vertices in a topological order and touching incident arcs.

Summing over all nontrivial SCCs yields a conservative bound of

$$O\left(n + m + \sum_S \left(m_S \log m_S + r_S(k + m_S) + \log(k)\, m_S\right)\right),$$

with the understanding that $|S|$ here denotes the vertex set of SCC S. When the entire graph is one nontrivial SCC, the implementation optionally runs both $L1$ and $L2$ orderings and keeps the better seed, doubling seed cost in that case only.

IPSNS initialization.

Initialization builds both seeds, evaluates their backward weights by scanning all m arcs once, and computes SCCs and SCC edge lists in $O(n+m)$. The cost is therefore the sum of one LR-TA run, one full WMSF-style seed run, and $O(m)$ objective evaluation.

IPSNS per-iteration cost.

One iteration scores every nontrivial SCC by scanning its internal arcs against the current rank vector, costing $O(m)$ overall. Selecting a SCC from the top-K positive contributors adds $O(s \log s)$ sorting, where s is the number of eligible SCCs, plus $O(K)$ weighted sampling. On the chosen SCC S, the destroy step touches only internal arcs; restricted cycle reduction performs at most c_S cycle searches and deactivations within S, each costing $O(n_S + m_S)$ in the worst case; restricted add-back sorts at most r_S candidates and may perform up to r_S reachability tests and topological recomputations on the SCC subgraph. The iteration finishes with one global topological sort and one full backward-weight scan, each $O(n + m)$. A conservative per-iteration bound is therefore

$$O(m + s \log s + c_S(n_S + m_S) + r_S \log r_S + r_S(n_S + m_S)).$$

Total IPSNS cost.

Let $T' \le T$ be the number of executed iterations. Then

$$O\Big(\text{seed costs} + T'\big(m + s \log s\big) + \sum_{t=1}^{T'} \big(c_{S_t}(n_{S_t} + m_{S_t}) + r_{S_t} \log r_{S_t} + r_{S_t}(n_{S_t} + m_{S_t})\big)\Big),$$

where S_t is the SCC selected at iteration t. This expression is intentionally conservative: SCC-local operations are not always cheaper than global ones, and large selected SCCs can dominate runtime.

Space complexity.

All three components store the graph in sparse edge-indexed form: endpoint arrays, original and mutable weights where needed, activity flags, and outgoing adjacency lists, requiring $O(n + m)$ space. Temporary DFS, reachability, and topological-sort structures also use $O(n)$ or $O(n + m)$ scratch memory. IPSNS additionally stores incumbent snapshots of the activity vector and removed-edge set, still within $O(n + m)$ overall for the global state; SCC-local repair uses temporary allowed-node and allowed-edge masks of size $O(n + m)$ but does not materialize dense matrices.

4.5 Algorithmic invariants

The most important invariant in the framework belongs to IPSNS rather than to LR-TA. Let $\pi^{(0)}$ denote the better of the two internal seeds and let $\pi^{(t)}$ denote the incumbent ordering after t accepted-or-rejected IPSNS iterations. Proposition 3 implies

$$\mathrm{bw}\left(\pi^{(t+1)}\right) \leq \mathrm{bw}\left(\pi^{(t)}\right) \leq \mathrm{bw}\left(\pi^{(0)}\right) \qquad \text{for all } t \geq 0. \tag{5}$$

This monotone non-degradation property is not an approximation guarantee, but it is a concrete algorithmic promise: the final IPSNS solution is never worse than the better internal seed under the same objective and nonnegative-weight assumptions.

LR-TA and the WMSF-style seed have their own feasibility invariants. Propositions 1 and 2 formalize LR-TA acyclicity and add-back correctness; the WMSF-style seed alternates removal and minimization steps while checking acyclic feasibility through the same add-back rules. IPSNS inherits those feasibility properties locally during SCC repair and adds the stronger incumbent-protection rule globally. That combination supports a narrow guarantee about refinement quality without claiming any new theoretical approximation ratio.

5 Experimental design

The experimental design follows a deliberately scoped claim strategy. The study evaluates a reproducible framework for nonnegative weighted directed graphs, using sparse benchmark instances as the primary setting and dense complete ordering instances as a transfer test.

Five empirical questions motivate the design. The first is whether the full refinement framework improves on its internal seeds without violating the non-degradation invariant. The second is which components materially contribute to the final solution quality. The third is how close the framework comes to exact optimization on instances where exact optimization is feasible. The fourth is how the framework compares with strong internal and external baselines on the main sparse benchmark. The fifth is how far the sparse-benchmark conclusions transfer to dense complete ordering instances. Table 3 summarizes how the five computational components are used to answer these questions.

5.1 Datasets

The study uses two public benchmark families with different roles in the argument.

Table 3: Overview of the five committed experimental components used in the manuscript. EXP1b checks seed safety on the full internal benchmark; EXP2 studies design choices; EXP3 validates small instances against exact optimization; EXP4 compares against external baselines on the primary sparse benchmark; and EXP5 tests transfer to dense LOLIB instances.

Experiment	Purpose	Instances	Main setting
EXP1b	Internal safety check for LR-TA, WMSF, and IPSNS	105	sparse weighted DIMACS digraphs
EXP2	Add-back, refinement, and iteration ablation	10	representative sparse subset
EXP3	Exact validation against bitmask dynamic programming	57	standard nonnegative instances with $n \leq 20$
EXP4	External baseline comparison on the primary claim-bearing benchmark	97	standard nonnegative sparse instances
EXP5	Dense transfer test and scope boundary analysis	50	LOLIB complete weighted ordering instances

The primary benchmark family comes from the public `graph-benchmarks` collection [23]. After path-level deduplication, this yields 105 unique sparse weighted DIMACS digraph instances. These instances support the internal benchmark, the ablation study, the exact small-instance validation, and the external sparse comparison.

The standard analysis is restricted to the nonnegative-weight subset. Eight instances are excluded from the standard sparse comparison set because they contain negative-weight arcs: `gerez`, `howard-max`, `k3_3`, `ku`, `peterson`, `peterson1`, `peterson2`, and `stg0`. This exclusion is methodological rather than cosmetic. The local-ratio construction and the interpretation of incumbent protection in this study are framed for nonnegative weights, so mixing negative-weight instances into the main tables would weaken the meaning of the guarantee. After exclusions, the primary sparse comparison set contains 97 standard instances.

The exact-validation subset comprises the 57 standard nonnegative benchmark instances with $n \leq 20$ included in the bitmask dynamic-programming study. Negative-weight instances and trivial $n = 0$ cases are excluded. This subset is not treated as a separate benchmark family; it is a validation slice used to judge heuristic quality against certified optima.

The dense transfer test uses a 50-instance subset of the Linear Ordering Problem Library (LOLIB) 2010 [21, 22]. These instances differ structurally from the sparse benchmark in a way that matters algorithmically. They are complete weighted ordering instances, and the selected subset spans three families: SGB (25 instances), IO (10 instances), and RandA1 (15 instances across $n = 100$, 150, and 200). In the converted DIMACS representation used in the experiments, every ordered pair carries weight, so these instances are effectively dense complete tournaments or dense linear-ordering inputs rather than sparse general digraphs.

This distinction is central to the scope of the study. The sparse benchmark is the main source of positive claims. LOLIB is not treated as coequal evidence for the primary contribution; it is used to determine where a sparse weighted-digraph heuristic remains competitive and where a dense-ordering-native ordering method becomes preferable.

The evaluation covers sparse weighted digraph benchmarks, exact small-instance validation, medium-scale MIP validation, external heuristics, a dense LOLIB transfer test, and one application-facing public ordering instance.

5.2 Algorithms and baselines

The evaluation includes the proposed refinement framework, its internal seed methods, an exact reference method for the small-instance study, and a set of external or simple baseline heuristics.

The full method of interest is IPSNS, which starts from the better of two internal seeds and applies incumbent-protected SCC-local refinement. The two seeds are LR-TA and WMSF. LR-TA is the engineered local-ratio plus topological add-back construction described in Section 4; WMSF is a weighted removeArcs–minimize–stabilize seed that is carried forward as a baseline and seed source.

For the exact small-instance validation, the reference method is exact bitmask dynamic programming. That solver is not used as a scalable comparator on the full sparse benchmark; its role is only to certify optimality on the small validation subset.

The sparse and dense comparison studies also include five additional baselines:

- the DRMacIver/FAS tool [14], an external deterministic matrix-based ordering heuristic that maximizes forward weight on pairwise evidence and is evaluated here under the common backward-weight objective;
- python-igraph's Eades-style feedback arc interface [13];
- a weighted Eades adaptation implemented locally and described explicitly as an adaptation rather than as the original [7] algorithm;
- Borda net-score ordering [15];
- random multistart ranking.

DRMacIver/FAS is not an exact method. Its matrix-based formulation makes it a particularly relevant comparator for dense LOLIB instances, which are complete weighted ordering problems, whereas the sparse benchmark exercises it only through a sparse entry-list interface.

The selection of each baseline follows a reproducibility and comparability criterion. Exact bitmask DP is the certified reference for instances small enough for exhaustive search; it validates heuristic quality without claiming that exact optimization scales globally. The time-capped MIP baseline (EXP8) extends certified reference to medium instances where bitmask DP is infeasible, reporting provably optimal solutions where the solver terminates within 120 s and treating time-limited cases as incomplete evidence. The python-igraph Eades-style implementation is selected because it is a documented library-grade baseline with a public API, representing what a practitioner could call with a single invocation and setting the reproducible practical floor. The weighted Eades adaptation is included to assess whether applying the Eades net-score

principle with arc weights closes the gap to IPSNS; it is labeled explicitly as an adaptation because the original [7] algorithm is unweighted. DRMacIver/FAS is included as an independent external method with a fundamentally different algorithmic model—matrix-based pairwise ordering rather than sparse-digraph cycle reduction—providing a cross-paradigm comparison and serving as the reference point for dense LOLIB performance. Borda net-score and random multistart serve as calibration anchors whose presence ensures the benchmark cannot be described as constructed against weak comparators. The adjacent-swap and single-vertex insertion local search comparisons (EXP7) directly test whether generic order-local moves without SCC structure can replicate IPSNS gains, isolating the contribution of SCC-aware destroy-repair from simpler greedy improvement.

This baseline mix is deliberate. It does not attempt to exhaust every recent ranking model or every exact solver. In particular, it does not include a learned model such as GNNRank [16], a memetic linear-ordering solver such as LOP_MA-EDM [24], or the recent minimal-and-stable weighted feedback arc set heuristic of Cavallaro and Cutello [17]. Those omissions are stated explicitly. The intended comparison surface is the ecosystem of deterministic or near-deterministic weighted-digraph heuristics and accessible external tools that a practitioner could reasonably invoke without building a separate learning or solver pipeline. Dense LOP-native solvers such as LOP_MA-EDM and the methods surveyed in [21] are not primary baselines because they are designed for the complete pairwise-matrix input model; applying them to sparse digraph instances would blur two structurally different problem regimes. These solvers are cited as the appropriate reference class for dense linear ordering but are not described as experimentally compared against the proposed framework on the sparse benchmark.

Several fairness rules govern the comparisons. First, all quality comparisons use the same backward-weight objective on the original nonnegative arc weights. Second, official external methods and local adaptations are distinguished explicitly. The weighted Eades method is an adaptation of the unweighted Eades idea [7]; it is not presented as an official external implementation. Third, DRMacIver/FAS is reported exactly as it was run in the study. The sparse comparison therefore retains its four incompletions on the standard set, and the dense comparison retains its strong overall performance. Finally, outcomes are interpreted in scope: losing to a dense-ordering-native method on dense LOLIB is not treated as evidence against the sparse-graph claim, and strong sparse performance is not used to imply dense linear-ordering dominance.

5.3 Evaluation metrics

The primary objective throughout the paper is backward weight (BW), defined in Section 3. Lower BW is always better, and all main tables are organized around that quantity.

Several secondary metrics support interpretation. The number of global-best instances records how often a method attains the minimum observed BW among the compared algorithms on a given benchmark. For the external sparse comparison, relative excess over IPSNS records the mean percentage by which a method's BW exceeds IPSNS on completed instances. For the exact small-instance study, the mean gap from

the certified optimum and the number of optimal matches summarize how close each heuristic comes to exact optimization. Mean runtime is reported for practical context, although it is not normalized across machines. The dense complete-ordering study also reports forward ratio as a secondary descriptive measure of how much pairwise weight is oriented forward by the returned ranking. Finally, the incumbent-violation count records how often IPSNS would return a solution worse than LR-TA or WMSF; under the intended invariant this count should remain zero on the nonnegative setting. To supplement tabular comparisons, paired statistical tests are computed on the common completed instances of the external sparse comparison, using per-instance backward weight as the outcome unit. The Wilcoxon signed-rank test and sign test characterize the distribution of per-instance paired differences; they are reported as distributional evidence for the observed outcome asymmetry and do not replace exact optimality certificates. A supplementary budget experiment (EXP6) runs IPSNS at six iteration budgets (10, 25, 50, 100, 200, and 400) on a 20-instance representative sparse subset to characterize how solution quality varies with the iteration budget relative to runtime cost. A generic local-search comparison (EXP7) tests whether adjacent-swap or single-vertex insertion local search seeded from LR-TA can replicate IPSNS quality on the same representative subset. A time-capped MIP baseline (EXP8) was also run on a deterministic 15-instance medium sparse subset ($n \leq 318$, 120 s per instance) to compare IPSNS with solver-certified optima where the MIP proves optimal. As an application-facing public ordering example, we convert the SNAP Wikipedia adminship vote network [25] into a weighted ordering instance by restricting to the top 50 users by vote degree and using vote counts as directed nonnegative arc weights (EXP9).

5.4 Implementation and reproducibility

All reported numbers are taken from saved experimental summaries and regenerated paper assets. Tables and figures in the results section are produced from the recorded JSON, CSV, and Markdown outputs of the five computational components, with key reported numbers checked against their source files before inclusion.

The experimental runs use fixed seeds where randomness exists and consistent wrappers for the compared methods. For IPSNS, the saved summaries record the default full WMSF seed mode and the iteration budgets used in each study. The ablation study is especially important here because it justifies those defaults conservatively: on the 10-instance subset, the mean BW for 50 iterations is identical to the 400-iteration mean, and the no-SCC-priority variant is nearly identical to the default. Those results are interpreted as subset-level support for reasonable defaults, not as universal parameter optimality.

Negative-weight sparse instances are excluded from the standard benchmark because they do not match the nonnegative setting studied here. DRMacIver/FAS incompletions on the sparse benchmark are reported explicitly rather than hidden in filtered tables. The dense LOLIB transfer test is retained because it documents where the framework remains competitive and where dense complete ordering instances favor a specialized external method.

Table 4: Comparison on the 97 standard nonnegative sparse instances used for the primary claim-bearing benchmark. BW denotes backward weight; lower values are better. Relative excess is the mean percentage by which an algorithm's BW exceeds IPSNS on completed instances.

Method	Complete	Mean BW	Mean RT (s)	Best	Relative excess
IPSNS (ours)	97/97	37,698	21.92	96	0.00%
LR-TA (ours)	97/97	38,327	0.08	80	0.71%
WMSF seed	97/97	40,005	1.31	61	2.06%
DRMacIver/FAS	93/97	53,173	4.00	56	21.61%
igraph Eades	97/97	95,920	0.01	40	30.54%
Weighted Eades	97/97	99,689	0.11	42	30.48%
Borda net score	97/97	512,277	0.00	27	55.55%
Random multistart	97/97	1,075,258	0.03	42	49.76%

DRMacIver/FAS is the strongest external comparator on this benchmark, but it completes 93 of 97 instances because two DAG cases return empty-tournament errors and two large sparse instances time out in the wrapper used here.

Runtime values are wall-clock times on a Linux workstation (Intel Core i7-12700K, 62 GiB RAM, Ubuntu 24.04, Python 3.12) and are used mainly for within-study comparison; external tools were invoked via their documented interfaces.

6 Computational results

The computational evidence should be read in the same order as the experimental design. The paper first establishes the main sparse-benchmark result, then checks exact small-instance quality, then uses ablations to interpret the engineering choices, and finally tests transfer to dense LOLIB instances. This ordering matters because the main claim is intentionally scoped: IPSNS is presented as a strong, reproducible heuristic framework for nonnegative sparse weighted directed graphs.

6.1 Sparse weighted graph benchmarks

The primary result comes from the standard nonnegative sparse benchmark, with the full internal benchmark supplying the safety check for the refinement stage. Table 4 summarizes the main comparison on 97 instances, and Figures 2 and 3 visualize the gap structure and the distribution of instance wins.

The main outcome is clear on this benchmark. IPSNS achieves the lowest observed backward weight among the tested methods on 96 of the 97 standard instances. Its mean BW is 37,698, compared with 38,327 for LR-TA and 40,005 for WMSF. Among the external methods, DRMacIver/FAS is the strongest comparator, but its mean BW is still 53,173 on the completed standard instances, or about 21.61% above IPSNS. DRMacIver/FAS completes 93 of 97 instances; the remaining four cases are two DAG inputs rejected by the wrapper and two large sparse instances that time out in the wrapper used here. Sparse-comparison averages for DRMacIver/FAS are computed on completed runs only; incompletions are reported separately and are not hidden. The

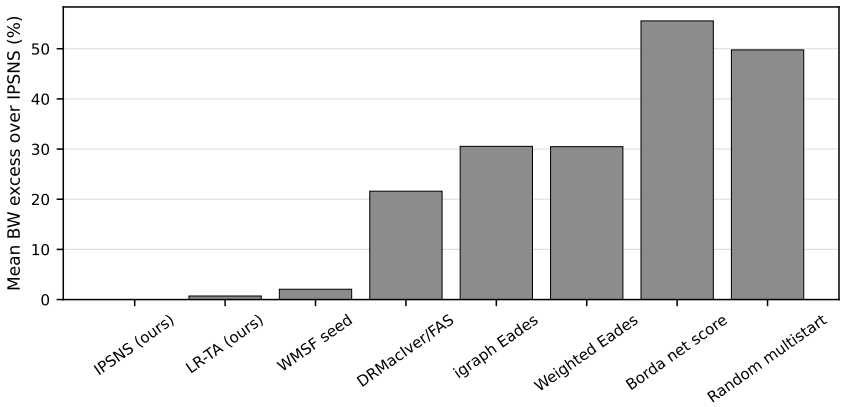

Fig. 2: Mean backward-weight excess over IPSNS on the 97 standard nonnegative sparse instances. Lower is better; IPSNS is the reference at zero.

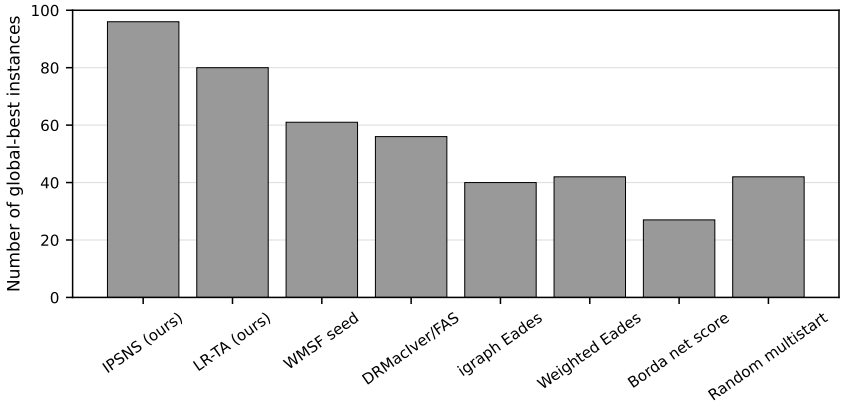

Fig. 3: Number of instances on which each method attains the lowest observed backward weight on the primary sparse benchmark.

remaining baselines are substantially weaker on average, with the Eades variants about 30% higher than IPSNS and the simpler Borda and random baselines far behind.

This result addresses baseline strength directly. The sparse-benchmark claim is not based only on a comparison among internal variants. It survives contact with a strong external comparator, a widely available graph-library heuristic, and several simple but informative ranking baselines. The most important external comparison is DRMacIver/FAS, not because it wins the benchmark, but because it remains competitive on much of the sparse set despite being a matrix-based pairwise-ordering heuristic rather than a sparse-digraph-native method. Even there, the gap is small only on one instance: `r20_60`, where DRMacIver/FAS beats IPSNS by 3 BW units. That same

instance is also the only IPSNS near-miss in the exact-validation study below, which makes the sparse story internally consistent rather than contradictory.

Paired statistical tests on the 93 standard instances completed by both IPSNS and DRMacIver/FAS show that the per-instance backward-weight differences favor IPSNS asymmetrically: IPSNS achieves a strictly lower backward weight on 37 instances, ties on 55, and is exceeded on 1. Both the Wilcoxon signed-rank test and the sign test yield $p < 0.001$, indicating that the observed win–loss asymmetry is unlikely under a null distribution of symmetric per-instance outcomes. These tests provide distributional support for the benchmark differences; they do not certify optimality. On the standard 97-instance nonnegative benchmark, IPSNS never loses to LR-TA (16 wins, 81 ties, zero losses) and never loses to WMSF (36 wins, 61 ties, zero losses), consistent with the non-degradation invariant. Table 5 summarizes the paired outcomes for all comparisons.

Table 5: Paired statistical tests comparing IPSNS against each baseline on common completed instances of the standard nonnegative sparse benchmark (negative-weight instances excluded). Improvement is the per-instance reduction in backward weight achieved by IPSNS (positive values indicate IPSNS attains a lower backward weight). The Wilcoxon signed-rank test and sign test are two-sided and provide distributional support for the observed differences; they do not replace exact optimality certificates.

Comparison	n	W	T	L	Med. ΔBW	p (Wilcoxon / Sign)
IPSNS vs LR-TA	97	16	81	0	0.00	< 0.001 / < 0.001
IPSNS vs WMSF	97	36	61	0	0.00	< 0.001 / < 0.001
IPSNS vs DRMacIver/FAS	93	37	55	1	0.00	< 0.001 / < 0.001
IPSNS vs igraph Eades	97	57	40	0	798	< 0.001 / < 0.001
IPSNS vs Weighted Eades	97	55	42	0	859	< 0.001 / < 0.001

W = wins (IPSNS lower BW), T = ties, L = losses. Med. ΔBW is the per-instance median improvement in backward-weight units. DRMacIver/FAS has 93 paired instances (four sparse incompletions and eight negative-weight instances excluded). IPSNS never loses to LR-TA or WMSF (zero losses), consistent with the non-degradation invariant.

The full internal benchmark strengthens this interpretation by showing that the refinement stage does not obtain these gains by occasionally damaging its seeds. Across 105 sparse benchmark instances, IPSNS is never worse than LR-TA and never worse than WMSF, with zero incumbent-protection violations in the verified summary. It strictly improves over LR-TA on 16 instances and over WMSF on 36 instances. The mean runtime per instance rises from 0.074 seconds for LR-TA and 1.24 seconds for WMSF to about 20.2 seconds for IPSNS, so the gain is not free; however, it is gained without giving up the best internal seed. That property connects the empirical behavior back to the monotone non-degradation invariant in Section 4.

The runtime pattern also clarifies the engineering tradeoff. LR-TA is the fastest strong method in the study: its mean sparse-benchmark runtime is only 0.08 seconds per instance, yet its mean BW is within about 1.7% of IPSNS. That makes LR-TA a credible low-latency choice when refinement time is limited. IPSNS then occupies the higher-quality point on the tradeoff curve, paying more runtime to secure the best

observed solutions on almost every instance. The study therefore provides a fast seed heuristic, a stronger refinement framework, and a clear statement of what the extra computation buys.

A supplementary budget study (EXP6) tests how IPSNS quality varies with the iteration budget on a 20-instance representative sparse subset (instances with EXP4 IPSNS runtime exceeding 60 s or fewer than 10 vertices excluded). Across six budgets from 10 to 400 iterations, the mean backward weight on this subset is identical (84,614 BW units) and no losses to LR-TA occur at any budget. The improvement over LR-TA is concentrated on 7 of the 20 instances and is secured within the first 10 iterations; additional budget does not change the outcome on this subset. Runtime scales approximately linearly: mean per-instance time rises from 0.28 s at 10 iterations to 4.40 s at 400 iterations. Table 6 and Figure 4 summarize these results.

Table 6: IPSNS quality-vs-runtime tradeoff across iteration budgets on a 20-instance representative sparse subset (EXP6). Mean backward weight (BW), mean runtime per instance, and win/tie/loss counts versus LR-TA from EXP4 are shown for each budget. Rel. excess is the mean percentage by which that budget's BW exceeds the 400-iteration result on the same subset; lower is better. LR-TA (from EXP4) is included as the low-latency reference.

Algorithm (budget)	Mean BW	Mean RT (s)	W / T / L vs LR-TA	Rel. excess vs 400-iter
LR-TA (seed only)	87,365	0.035	— / — / —	—
IPSNS (10 iters)	84,614	0.275	7 / 13 / 0	0.00%
IPSNS (25 iters)	84,614	0.425	7 / 13 / 0	0.00%
IPSNS (50 iters)	84,614	0.698	7 / 13 / 0	0.00%
IPSNS (100 iters)	84,614	1.222	7 / 13 / 0	0.00%
IPSNS (200 iters)	84,614	2.286	7 / 13 / 0	0.00%
IPSNS (400 iters)	84,614	4.398	7 / 13 / 0	0.00%

Subset: 20 representative sparse instances with EXP4 IPSNS runtime ≤ 60 s and $n \geq 10$, spanning density from very sparse to moderately dense. W = wins (IPSNS lower BW than LR-TA), T = ties, L = losses (none observed). Rel. excess: mean percentage above the 400-iteration result on the same instance set.

EXP7 asks whether IPSNS improvements over LR-TA can be matched by simpler order-local improvement heuristics. On the 18 non-negative instances of the representative subset, adjacent-swap local search (accepting any adjacent pair swap that reduces backward weight) finds no improvements: LR-TA already constitutes an adjacent-swap local optimum on every instance. Single-vertex insertion local search (finding the globally best position for each vertex in turn) improves over LR-TA on 4 instances and achieves mean backward weight 95,779, but it still loses to IPSNS on 5 instances, all large sparse graphs ($n \geq 1{,}000$). Using the better of LR-TA and WMSF as the insertion seed narrows the gap slightly (mean BW 95,617) but leaves 3 losses to IPSNS. The instances where insertion LS falls short overlap directly with the highest-gain IPSNS instances in EXP4. Table 7 reports the comparison.

The sparse results cover a broad but intentionally scoped benchmark. The comparison uses 97 standard nonnegative sparse instances after exclusions and includes

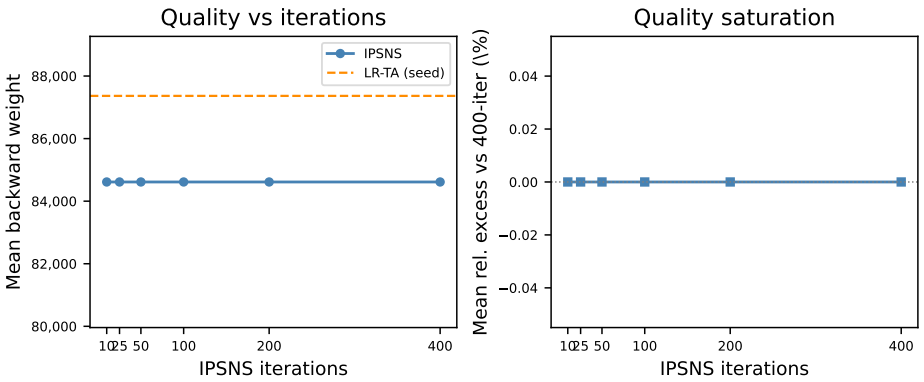

Fig. 4: IPSNS quality and saturation on the 20-instance representative sparse subset (EXP6). *Left:* mean backward weight versus iteration budget; LR-TA (dashed) is the low-latency reference. *Right:* mean relative excess above the 400-iteration result; zero at all tested budgets indicates complete quality saturation from the first budget tested on this subset.

Table 7: Generic order-local improvement versus IPSNS on the 18 non-negative instances of the EXP7 representative sparse subset (peterson1 and peterson2 are excluded due to negative arc weights). Adj-swap LS and insertion LS are seeded from LR-TA; best-seed insertion LS uses the better of LR-TA and WMSF as the starting point. W / T / L counts are against the 400-iteration IPSNS result from EXP4.

Method	Mean BW	Mean RT (s)	W / T / L vs LR-TA	W / T / L vs IPSNS
LR-TA (seed)	97,185	—	—	—
IPSNS (EXP4 full)	94,127	—	—	—
Adj-swap LS from LR-TA	97,185	0.026	0 / 18 / 0	0 / 11 / 7
Insertion LS from LR-TA	95,779	0.234	4 / 14 / 0	1 / 12 / 5
Insertion LS from best seed	95,617	0.383	7 / 11 / 0	1 / 14 / 3

Adj-swap LS stops at local optimum (max 20 passes); insertion LS stops at local optimum, 200 accepted moves, or 60 s per instance. Both seeded methods ran from LR-TA for comparability with IPSNS's starting seed. Negative-weight instances (peterson1, peterson2) are excluded from this table.

eight algorithms in total. These results are specific to this setting and do not extend to negative-weight cases or dense complete ordering benchmarks.

6.2 Exact validation and ablation evidence

The exact small-instance validation and the ablation study serve different purposes, but together they explain why the framework is both credible and interpretable. Table 8 reports the exact validation results, and Table 9 reports the component ablation.

Table 8: Exact validation on the 57 standard nonnegative instances with $n \leq 20$. Mean gap is measured against exact bitmask dynamic programming on each instance.

Method	Optimal	Optimality rate	Mean gap
IPSNS (ours)	56/57	98.2%	0.0006%
LR-TA (ours)	55/57	96.5%	0.0590%
WMSF seed	51/57	89.5%	0.0961%

This table supports empirical near-optimality on small instances only; it does not imply a general approximation guarantee. The only IPSNS near-miss is instance `r20_60`.

Table 9: Ablation study on 10 representative sparse instances. Relative change is measured against LR-TA. Negative percentages indicate improvement.

Variant	Mean BW	Relative change	Mean RT (s)
LR without add-back	4525.1	+5.94%	0.017
LR-TA	4271.5	+0.00%	0.047
WMSF seed	4332.5	+1.43%	0.040
Best seed, no refinement	4271.5	+0.00%	0.069
IPSNS, 50 iterations	4239.2	-0.76%	0.778
IPSNS, 400 iterations (default)	4239.2	-0.76%	5.734
IPSNS, no SCC priority	4239.1	-0.76%	5.727

The add-back phase accounts for the largest mean improvement (about 5.9%), while IPSNS contributes a further 0.75% on this subset. The equal 50- and 400-iteration means support the default iteration budget only as subset-level evidence, not as a universal tuning rule.

The exact small-instance study supports a quality-validation narrative rather than a theoretical one. On the 57 standard nonnegative instances with $n \leq 20$ included in the dynamic-programming study, IPSNS matches the certified optimum on 56 instances and has a mean gap of 0.0006% from the exact optimum. LR-TA matches the optimum on 55 instances with mean gap 0.0590%, and WMSF matches the optimum on 51 with mean gap 0.0961%. These numbers show that the framework is not merely competitive in relative benchmark terms; on the exact-validation subset, it is essentially exact in practice.

This table does not provide an approximation guarantee, a worst-case bound, or a proof of global behavior beyond the validated subset. The appropriate conclusion is that IPSNS is empirically near-optimal on the exact-validation subset. The only IPSNS near-miss is instance `r20_60`.

The exact-validation result also helps interpret the single sparse-benchmark loss to DRMacIver/FAS. The only IPSNS near-miss is again `r20_60`. This is also the only standard small instance where IPSNS fails to match exact optimum. That consistency reduces the chance that the sparse comparison is driven by unstable reporting or

Table 10: Time-capped MIP baseline (EXP8) on 15 selected medium sparse instances ($n \leq 318$, 120 s time limit per instance). The solver certifies optimal solutions on 7 instances; the remaining 8 exceed the time limit and yield no MIP incumbent. IPSNS gap is measured against the solver-certified optimum on the 7 proven-optimal instances only.

Metric	Value
Instances selected	15
Proven optimal (MIP solved)	7
Time-limited (no incumbent)	8
Time limit	120 s
IPSNS matches MIP optimum	6/7
IPSNS mean gap (proven-optimal)	0.025%
IPSNS max gap (proven-optimal)	0.178%

artifact mismatch. Instead, it suggests a small, structurally difficult case where the framework is near optimum but not uniquely dominant.

A supplementary time-capped MIP baseline (EXP8) extends this exact-validation evidence to medium sparse instances where bitmask dynamic programming is no longer feasible. On 15 selected instances with $n \leq 318$ and a 120 s per-instance time limit, the HiGHS solver certifies optimal solutions for 7 instances; the remaining 8 exceed the time limit and yield no MIP incumbent. On the 7 proven-optimal instances, IPSNS matches the certified optimum on 6; the single exception is again `r20_60`, where IPSNS is 0.178% above the MIP optimum. The 8 time-limited cases provide no gap certification and are treated as incomplete solver evidence only. Table 10 summarizes the aggregate outcomes.

The application-facing public ordering example (EXP9) gives a deliberately different picture: on the dense vote-degree subgraph (n=50, m=752, density = 0.307), DRMacIver/FAS achieves backward weight 135 while IPSNS ties LR-TA at 141, consistent with the structural boundary observed on LOLIB (Table 11).

The ablation study explains where the framework's gains come from. The add-back phase is the dominant contributor among the tested design choices. Removing it raises mean BW from 4,271.5 for LR-TA to 4,525.1 for the no-add-back variant, a degradation of about 5.94%. This is the clearest experimental support for treating topological add-back as a substantive engineering contribution rather than a decorative post-processing step.

The refinement stage then provides a smaller but still measurable gain. Moving from LR-TA to IPSNS with 50 iterations reduces mean BW from 4,271.5 to 4,239.2, about 0.76%. In absolute terms that gain is much smaller than the add-back gain, but it is still meaningful because it is obtained after starting from a strong seed and while preserving the non-degradation invariant. This is the kind of marginal improvement

Table 11: Application-facing public ordering example: Wikipedia adminship vote network (EXP9). Lower backward weight is better. The instance is the top-50 vote-degree subset of the SNAP wiki-Vote network [25] converted to a nonnegative weighted directed graph (n=50, m=752, density = 0.307).

Algorithm	BW	Runtime (s)
DRMacIver/FAS	135	0.79
LR-TA (ours)	141	0.23
IPSNS (ours)	141	0.72
WMSF seed	142	0.24
igraph Eades	153	0.31
Weighted Eades	157	0.02

one expects from a bounded refinement layer: it does not re-invent the seed; it improves the hard residue that remains after seeding.

The ablation results also provide the main evidence for parameter justification, and the interpretation must remain conservative. On this 10-instance subset, the add-back phase accounts for the largest mean improvement (about 5.9%), while IPSNS contributes a further 0.75%. The 50-iteration and 400-iteration IPSNS variants have the same mean BW, which suggests that the default budget is already beyond the point of visible average improvement on these instances. Likewise, removing SCC priority changes the mean only from 4,239.2 to 4,239.1, effectively negligible at the reported precision. These observations support the default settings as reasonable choices, but they do not show that those settings are universally optimal on other graph families or scales.

6.3 Dense LOLIB transfer test

Table 12 and Figure 5 summarize the dense LOLIB transfer test.

This result defines an important scope boundary. On the 50 dense complete ordering instances, DRMacIver/FAS is the best overall method in the study: it wins 45 instances and achieves mean BW 571,687, compared with 582,354 for IPSNS. The mean gap is 3.88% in DRMacIver/FAS's favor. IPSNS remains second overall and retains zero incumbent violations against LR-TA and WMSF, but the dense benchmark favors the matrix-based comparator.

The family breakdown explains why. DRMacIver/FAS wins 24 of the 25 SGB instances and all 15 RandA1 instances. IPSNS is more competitive on IO, where it wins 4 of 10 against DRMacIver/FAS's 6 of 10, but that does not overturn the overall dense-ordering result. The cleanest interpretation is structural: DRMacIver/FAS is a matrix-based pairwise-ordering heuristic whose documented input model aligns

Table 12: Dense LOLIB transfer test on 50 complete weighted ordering instances. Lower BW is better. The upper panel reports overall performance; the lower panel isolates the two most relevant methods by family to make the dense-scope boundary explicit.

Method	Mean BW	Best	Mean RT (s)
DRMacIver/FAS	571,687	45/50	1.01
IPSNS (ours)	582,354	5/50	37.92
LR-TA (ours)	582,948	2/50	0.22
WMSF seed	585,926	1/50	0.19
igraph Eades	659,570	0/50	0.01
Weighted Eades	675,235	0/50	0.01
Random multistart	883,664	0/50	0.02
Borda net score	1,066,045	0/50	0.00

Family breakdown for IPSNS and DRMacIver/FAS

Family	IPSNS best	DRMacIver/FAS best	Mean BW (IPSNS / DRMacIver/FAS)
SGB	1/25	24/25	1,048,056 / 1,036,085
IO	4/10	6/10	36,996 / 32,860
RandA1	0/15	15/15	169,755 / 156,909

DRMacIver/FAS is tournament-native and is the best overall dense LOLIB method in this study (45/50 instances, 3.88% lower mean BW than IPSNS). EXP5 is therefore a transfer test and scope boundary, not evidence for universal dominance on dense linear-ordering problems.

with LOLIB's complete weighted ordering format, whereas IPSNS is designed around sparse weighted-digraph seeding and SCC-local refinement. Dense complete ordering remains an active research area [19, 26], and when every ordered pair carries weight the advantage shifts toward methods specialized for matrix-based pairwise evidence.

This finding sharpens the overall claim. Retaining the dense transfer study shows that the framework transfers nontrivially to a dense benchmark, remains competitive on a structured IO subset, and still preserves the internal no-worsening invariant, but dense complete ordering is not the framework's strongest regime.

Across the section as a whole, the empirical message is coherent. The strongest claim belongs to the sparse nonnegative benchmark, where IPSNS attains the best observed backward weight among the tested methods on 96 of 97 standard instances and never returns a result worse than the better internal seed. The exact-validation subset shows that the quality story is not merely relative to heuristics, and the ablation study shows that the gains are not coming from an uninterpretable black box. The dense transfer study then marks the boundary of that success case. The framework is a strong and reproducible method for the sparse weighted-digraph regime, and the computational evidence shows how far that statement can be pushed.

7 Discussion

The computational evidence supports a clear but bounded interpretation of the proposed framework. The strongest result is the sparse nonnegative benchmark, where IPSNS achieves the lowest observed backward weight among the tested methods on 96

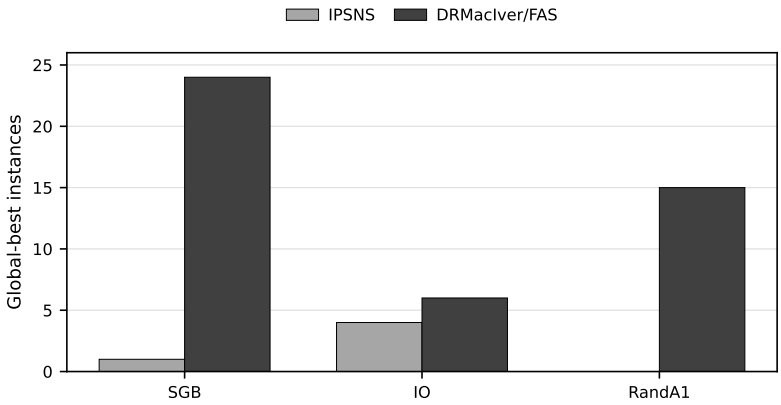

Fig. 5: Global-best counts by LOLIB family for IPSNS and DRMacIver/FAS.

of 97 standard instances and never degrades the better internal seed. The importance of that result is not only that the framework performs well, but that it performs well in a way that matches the intended algorithmic design. The methodology combines two complementary global constructions with a refinement stage that concentrates effort on the cyclic parts of the graph and accepts a move only when the global objective improves. The sparse results therefore do not look accidental: they are consistent with the structure that the method is designed to exploit.

The sparse benchmark interpretation begins with the role of the seed methods. LR-TA and WMSF produce feasible orderings by different mechanisms. LR-TA is driven by local-ratio reductions and topological add-back, whereas WMSF follows a removal-and-minimization template. On many sparse instances those two constructions are already strong, and LR-TA in particular offers an excellent quality–runtime tradeoff. IPSNS improves on that base by focusing its neighborhood moves on strongly connected components that still contribute substantial backward weight under the incumbent ordering. This is a natural strategy for sparse cyclic graphs. When the graph contains large acyclic regions and a smaller cyclic core, indiscriminate perturbation is wasteful; the search budget is better spent where ordering ambiguity remains concentrated. The empirical advantage of IPSNS on the sparse benchmark is therefore not simply that it runs longer than LR-TA. It is that the extra computation is directed at the right structural target.

Structural diagnostics computed from the converted DIMACS instance files confirm this contrast quantitatively. The standard sparse benchmark instances have a mean directed-graph density of 0.348 and a mean of 78.0% of vertices residing in nontrivial strongly connected components, with the SCC structure distributed across multiple components on most instances. The dense LOLIB instances, by contrast, have a mean density of 0.454 and essentially all vertices (98.7% on average) concentrated in a single large SCC, which is consistent with the complete pairwise-ordering matrix

structure. On the sparse benchmark, the IPSNS advantage over DRMacIver/FAS is negatively correlated with instance density (Spearman $r = -0.83$, $p < 0.001$), consistent with the expectation that matrix-based pairwise-ordering methods become more competitive as graph density increases toward the complete-ordering regime. These diagnostics are consistent with the intended operating regime of IPSNS; they do not replace controlled structural experiments. Table 13 summarizes the structural comparison.

Table 13: Structural comparison between the standard sparse benchmark and the dense LOLIB transfer benchmark. Density is $m/n(n-1)$ for directed graphs. Largest-SCC fraction is the fraction of vertices belonging to the largest strongly connected component. All statistics are computed from the converted DIMACS instance files used in the experiments.

Feature	Sparse benchmark	Dense LOLIB
Instances	95	50
Mean n (vertices)	869	94
Mean m (arcs)	1522	5083
Mean density	0.3477	0.4542
Mean largest-SCC frac.	0.6437	0.9867
Mean frac. in nontrivial SCCs	0.7796	0.9867
Mean nontrivial SCC count	11.2	1.0

Sparse benchmark uses the `graph-benchmarks` collection; statistics cover the standard nonnegative subset (negative-weight instances excluded) for which DIMACS files were located. LOLIB dense benchmark uses 50 converted DIMACS instances across three families (SGB, RandA1, IO). SCC features computed from converted DIMACS instance files.

The internal safety result is central to this interpretation. On the full 105-instance internal benchmark, IPSNS never returns a solution worse than LR-TA or WMSF. That is more than a convenient side condition. In any practical ordering pipeline, a refinement stage that occasionally damages a good seed is hard to trust and difficult to diagnose. The monotone non-degradation rule makes the framework traceable. If refinement finds no helpful move, the best seed is preserved. If refinement improves the incumbent, that improvement is directly visible in the final objective. This kind of operational reliability is more useful than a loosely justified claim that a local search method is "usually better."

The exact-validation subset strengthens the quality story without changing its theoretical status. IPSNS matches the certified optimum on 56 of 57 standard nonnegative instances in the validation subset and has a mean gap of 0.0006%. That is strong evidence that the heuristic framework is solving the small instances almost perfectly in practice. It is not, however, evidence for a new approximation ratio or a new optimality guarantee beyond the validated subset. When exact optimization is feasible, the framework tracks the optimum closely; when the instances become larger, the study relies on comparative benchmark evidence rather than on a theoretical claim it does not possess. The medium MIP subset (EXP8) strengthens the exact-validation evidence: where the solver proves optimal, IPSNS matches 6 of 7 certified optima,

while the 8 larger cases exceed the 120 s time limit and are treated as incomplete solver evidence rather than IPSNS failures.

The ablation study helps clarify which parts of the framework are carrying the result. The topological add-back phase is the dominant design element, with a mean backward-weight reduction of about 5.9% relative to local-ratio without add-back. The refinement stage then contributes a smaller additional improvement, about 0.75% on the ablation subset. On the full 105-instance sparse set, IPSNS strictly improves over LR-TA on 16 instances (mean improvement 581 BW units; median 0), confirming that the gains are concentrated on a harder subset; the runtime cost is the approximately 270-fold increase from LR-TA's 0.08 s mean to IPSNS's 21.9 s mean per instance. The supplementary budget study (EXP6) confirms that quality saturates within the first 10 SCC-local iterations; additional budget increases runtime proportionally without changing the mean backward weight. The generic local-search comparison (EXP7) reinforces this: adjacent-swap LS finds no improvements over LR-TA, and single-vertex insertion LS recovers only a fraction of IPSNS's advantage on the largest instances, confirming that SCC-local destroy-repair addresses cyclic sub-structure that sequential order-local moves do not systematically reach. The near-equality between 50 and 400 IPSNS iterations supports the default budget as a reasonable and stable choice on the tested subset, though not a universally optimal default for all graph families.

The dense LOLIB transfer test is the key scope boundary. On that benchmark, DRMacIver/FAS is stronger overall, winning 45 of 50 instances and outperforming IPSNS in mean backward weight. This result is part of the scientific contribution because it shows where the framework stops being the most competitive option among the methods tested. The structural explanation is straightforward. DRMacIver/FAS is a deterministic matrix-based pairwise-ordering heuristic, and LOLIB supplies dense complete ordering instances where every ordered pair contributes information. IPSNS, by contrast, is built around sparse weighted-digraph seeding and SCC-local refinement. Recent dense-ordering feedback arc set methods [19, 26] reinforce that complete weighted ordering is a distinct algorithmic regime rather than a mere scale change from sparse digraphs. When the graph is fully dense, the distinction between cyclic core and acyclic background becomes much less informative, and the advantage shifts toward methods specialized for matrix-based pairwise evidence. The application-facing public ordering example (EXP9) independently replicates this boundary: on the dense vote-degree subgraph (density 0.307), DRMacIver/FAS again obtains the lowest backward weight while IPSNS ties LR-TA, consistent with the dense-ordering regime observed on LOLIB.

The dense result is best interpreted as a boundary rather than a failure of transfer. IPSNS remains second overall on LOLIB, preserves its non-degradation property, and is competitive on the IO family, where it wins 4 of 10 instances. Dense complete ordering is not the paper's primary target, and a matrix-based pairwise-ordering heuristic is preferable there. It should also be noted that dedicated linear-ordering solvers such as LOP_MA-EDM [24] and the memetic methods surveyed in [21] were not rerun for this study; they may achieve lower backward weights than DRMacIver/FAS on LOLIB, reinforcing the conclusion that dense complete ordering instances favor methods specialized for that regime. Identifying where the methodology is strong, where

it is competitive, and where a different structural assumption favors another class of methods improves the credibility of the overall claim.

Several limitations remain and should be stated directly. First, the paper does not offer a new approximation guarantee; the local-ratio principle used in LR-TA is prior art, and the present contribution is an engineered reproducible instantiation plus the incumbent-protected SCC refinement framework. Second, the main claims apply only to nonnegative-weight instances; negative-weight cases are excluded as they do not match the framework's intended setting. Third, the exact-validation study is limited to small instances where bitmask dynamic programming is feasible and does not remove the need for heuristic benchmarking on larger graphs. Fourth, dense linear-ordering instances are not the primary target, and the LOLIB results show that clearly.

The paper is best read as an algorithm-engineering contribution to the minimum weighted feedback arc set problem on sparse directed graphs. A methodology that improves only when the global objective truly decreases and remains reproducible under fixed seeds is practically useful in any setting where ordering quality and deployability matter. The sparse benchmark results confirm this: the framework performs strongly across a heterogeneous collection of nonnegative sparse digraphs, with LR-TA/WMSF/IPSNS outperforming all tested baselines on 96 of 97 instances.

Future work should follow from those boundaries. One direction is stronger dense-native adaptation, pairing the non-degradation refinement discipline with a matrix-based or tournament-aware seed to extend competitive performance beyond the sparse regime. A second is broader evaluation on additional sparse weighted digraph families, including application-derived instances with heterogeneous edge-weight semantics. A third is parallel or asynchronous SCC refinement, exploiting the structural independence of disjoint SCC neighborhoods to improve scalability on large sparse instances. A fourth is exact or ILP-based polishing on medium-sized instances, to sharpen the quality reference point between the small exact-validation regime and the full sparse benchmark. A fifth is broader study of negative-weight or mixed-sign settings under an objective reformulation that preserves a clean interpretation of the refinement invariant.

Overall, the study is strongest when it remains faithful to its empirical boundaries. It presents a reproducible methodology for the minimum weighted feedback arc set problem in nonnegative weighted directed graphs, demonstrates clear strength on the main sparse benchmark, and documents where a different structural regime favors a different algorithmic family.

8 Conclusions

This paper studied the minimum weighted feedback arc set problem through its ordering interpretation on nonnegative weighted directed graphs. The main contribution is a reproducible computational methodology rather than a new approximation framework. It combines an engineered local-ratio construction with topological add-back, a complementary weighted seed, and an incumbent-protected SCC-local refinement stage. Within that design, LR-TA serves as a strong constructive seed, while IPSNS

is the main refinement framework that improves the incumbent only when the global backward-weight objective decreases.

The computational evidence supports a clear scoped conclusion. On the standard nonnegative sparse benchmark, IPSNS attains the best observed backward weight among the tested methods on 96 of 97 instances and improves on all tested internal and external comparators, including the strongest external baseline. On the full internal benchmark, the refinement stage never worsens the better internal seed, which confirms the intended non-degradation behavior in practice.

The exact-validation subset shows that the framework is near-optimal when bitmask dynamic programming certifies optima. The ablation study shows that the topological add-back phase is the dominant design element, with SCC-local refinement providing an additional but smaller gain.

The results also define an important boundary. On dense complete LOLIB ordering instances, the matrix-based pairwise-ordering baseline DRMacIver/FAS is stronger overall. This clarifies the regime in which the proposed framework is strongest: sparse nonnegative weighted digraphs. The method should be interpreted as a general weighted-digraph heuristic rather than as a dense-native linear-ordering solver.

The practical value of the framework lies in this combination of quality, structural focus, and reproducibility. The methodology is suitable for weighted ordering and cyclic-dependency repair settings where exact optimization is too expensive, sparse graph structure is meaningful, and refinement should not be allowed to damage a good seed. Within the stated boundaries, engineered local-ratio seeding plus incumbent-protected SCC refinement is an effective strategy for sparse weighted directed ordering problems.

Several directions follow from this evidence. The first is broader evaluation on additional sparse weighted digraph families, especially application-derived instances with heterogeneous edge-weight semantics. The second is stronger dense-native adaptation: the same incumbent-protected refinement discipline could be combined with a matrix-based or tournament-aware seed to extend competitive performance to dense complete ordering instances. The third is parallel or asynchronous SCC refinement, exploiting the structural independence of disjoint SCC neighborhoods to improve scalability on large sparse instances. The fourth is exact or ILP-based polishing on medium-sized instances, to close the quality reference gap between the small exact-validation subset and the full sparse benchmark.

Statements and Declarations

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Competing interests

The author declares that there are no known competing financial or non-financial interests that could have appeared to influence the work reported in this paper.

Author contributions

Soroush Vahidi: Conceptualization, Methodology, Software, Formal analysis, Investigation, Validation, Data curation, Visualization, Writing – original draft, Writing – review & editing.

Data and code availability

The code, experiment scripts, manifests, and processed result summaries for this study will be provided as supplementary material in Online Resource 1. Public benchmark instances are cited in the manuscript, and the supplementary material will document the conversion and preprocessing steps used in this study.

Generative AI and AI-assisted technologies

During the preparation of this work, the author used AI-assisted tools, including ChatGPT, Codex, Claude, and Perplexity AI, to support literature exploration, organization of material, language editing, and coding assistance. The author reviewed and edited all outputs, verified the relevant sources and experimental results, and takes full responsibility for the content of the submitted manuscript.

References

[1] Karp, R.M.: Reducibility among combinatorial problems. In: Complexity of Computer Computations, pp. 85–103. Springer, Boston, MA (1972). https://doi.org/10.1007/978-1-4684-2001-2_9

[2] Ailon, N., Charikar, M., Newman, A.: Aggregating inconsistent information: Ranking and clustering. Journal of the ACM **55**(5), 23–12327 (2008) https://doi.org/10.1145/1411509.1411513

[3] Flood, M.M.: Exact and heuristic algorithms for the weighted feedback arc set problem: A special case of the skew-symmetric quadratic assignment problem. Networks **20**(1), 1–23 (1990) https://doi.org/10.1002/net.3230200102

[4] Baharev, A., Schichl, H., Neumaier, A., Achterberg, T.: An exact method for the minimum feedback arc set problem. ACM Journal of Experimental Algorithmics **26**, 1–411428 (2021) https://doi.org/10.1145/3446429

[5] Bar-Yehuda, R., Geiger, D., Naor, J.S., Roth, R.M.: Approximation algorithms for the feedback vertex set problem with applications to constraint satisfaction and bayesian inference. SIAM Journal on Computing **27**(4), 942–959 (1998) https://doi.org/10.1137/S0097539796305109

[6] Demetrescu, C., Finocchi, I.: Combinatorial algorithms for feedback problems in directed graphs. Information Processing Letters **86**(3), 129–136 (2003) https://doi.org/10.1016/S0020-0190(02)00491-X

[7] Eades, P., Lin, X., Smyth, W.F.: A fast and effective heuristic for the feedback arc set problem. Information Processing Letters **47**(6), 319–323 (1993) https://doi.org/10.1016/0020-0190(93)90079-O

[8] Lempel, A., Cederbaum, I.: Minimum feedback arc and vertex sets of a directed graph. IEEE Transactions on Circuit Theory **13**(4), 399–403 (1966) https://doi.org/10.1109/TCT.1966.1082620

[9] Even, G., Naor, J.S., Schieber, B., Sudan, M.: Approximating minimum feedback sets and multicuts in directed graphs. Algorithmica **20**(2), 151–174 (1998) https://doi.org/10.1007/PL00009191

[10] Hecht, M., Gonciarz, K., Horvát, S.: Tight localizations of feedback sets. ACM Journal of Experimental Algorithmics **26**, 1–311319 (2021) https://doi.org/10.1145/3447652

[11] Brandenburg, F.J., Hanauer, K.: Sorting heuristics for the feedback arc set problem. In: WALCOM: Algorithms and Computation. Lecture Notes in Computer Science, vol. 7748, pp. 1–12. Springer, Berlin, Heidelberg (2013)

[12] Eades, P., Lin, X.: A heuristic for the feedback arc set problem. Australasian Journal of Combinatorics **12**, 15–25 (1995)

[13] python-igraph developers: python-igraph documentation: `Graph.feedback_arc_set`. https://python.igraph.org/en/stable/api/igraph.Graph.html. Documentation for the weighted feedback arc set API; accessed 2026-06-06 (2026)

[14] DRMacIver: Feedback-Arc-Set. https://github.com/DRMacIver/Feedback-Arc-Set. Software repository; experiment commit 16ff24a92fde886e58819180a9fe686e60991c5c; accessed 2026-06-06 (2026)

[15] Coppersmith, D., Fleischer, L.K., Rurda, A.: Ordering by weighted number of wins gives a good ranking for weighted tournaments. ACM Transactions on Algorithms **6**(3), 55–15513 (2010) https://doi.org/10.1145/1798596.1798608

[16] He, Y., Gan, Q., Wipf, D., Reinert, G.D., Yan, J., Cucuringu, M.: Gnnrank: Learning global rankings from pairwise comparisons via directed graph neural networks. In: Proceedings of the 39th International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 162, pp. 8581–8612 (2022). https://proceedings.mlr.press/v162/he22b.html

[17] Cavallaro, C., Cutello, V.: An efficient heuristic algorithm to compute minimal and stable weighted feedback arc sets. In: Proceedings of the International Conference on Software Engineering and Knowledge Engineering (SEKE), pp. 84–87 (2025). https://doi.org/10.18293/SEKE2025-049

[18] Simpson, M., Srinivasan, V., Thomo, A.: Efficient computation of feedback arc set at web-scale. Proceedings of the VLDB Endowment **10**(3), 133–144 (2016) https://doi.org/10.14778/3021924.3021930

[19] Fomin, F.V., Lokshtanov, D., Raman, V., Saurabh, S.: Fast local search algorithm for weighted feedback arc set in tournaments. Proceedings of the AAAI Conference on Artificial Intelligence **24**(1), 65–70 (2010) https://doi.org/10.1609/aaai.v24i1.7557

[20] Alon, N., Lokshtanov, D., Saurabh, S.: Fast FAST. In: Automata, Languages and Programming, pp. 49–58. Springer, Berlin, Heidelberg (2009)

[21] Marti, R., Reinelt, G., Duarte, A.: A benchmark library and a comparison of heuristic methods for the linear ordering problem. Computational Optimization and Applications **51**(3), 1297–1317 (2012) https://doi.org/10.1007/s10589-010-9384-9

[22] Marti, R.: LOLIB: Linear Ordering Problem Library. https://www.uv.es/~rmarti/paper/lop.html. Official library page; accessed 2026-06-06 (2010)

[23] Ali Dasdan: graph-benchmarks: Directed Graph Benchmarks in DIMACS Graph Format. https://github.com/alidasdan/graph-benchmarks. Dataset repository; accessed 2026-06-06 (2026)

[24] Carlos Segura: LOP_MA-EDM: Memetic Algorithms with Explicit Diversity Management for the Linear Ordering Problem. https://github.com/carlossegurag/LOP_MA-EDM. Software repository; accessed 2026-06-06 (2026)

[25] Leskovec, J., Huttenlocher, D., Kleinberg, J.: Predicting positive and negative links in online social networks. In: Proceedings of the 19th International Conference on World Wide Web. WWW '10, pp. 641–650. ACM, New York, NY, USA (2010). https://doi.org/10.1145/1772690.1772756

[26] Ostovari, M., Zarei, A.: A better LP rounding for feedback arc set on tournaments. Theoretical Computer Science **1015**, 114768 (2024) https://doi.org/10.1016/j.tcs.2024.114768